



Time Series Change Point Detection in R: Harbinger Methods



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Roadmap

This tutorial series walks through methods for detecting structural changes in time series, progressing from the simplest case to the most specialized. The journey begins with the intuition of a single breakpoint and advances toward more nuanced analytical perspectives, including volatility modeling, Bayesian monitoring, and ensemble aggregation.

01

Standard Workflow

The shared pipeline — data → fit → detect → evaluate → plot — reused across all methods

03

Structural Break Tests

Chow: regression-based single break test · **GFT**: global fluctuation monitoring

05

Volatility & Local Window

GARCH: volatility regime shifts · **SCP**: local window contrast

07

Bayesian Online Monitoring

BOCPD: Bayesian online changepoint detection

09

Univariate Drift Detectors

Page-Hinkley: cumulative mean shift test · **KSWIN**: distributional change via KS test

02

Classical Breaks

AMOC: single dominant break · **BinSeg**: greedy recursive segmentation · **PELT**: optimal partitioning with pruning

04

ChangeFinder Variants

CF-LR: linear regression prediction error · **CF-ARIMA**: ARIMA prediction error · **CF-ETS**: ETS prediction error

06

Adaptive Waypoint Models

Waypoint ED: feed-forward autoencoder · **Waypoint LSTM**: LSTM autoencoder · **Waypoint Conv**: convolutional autoencoder

08

Ensemble Support

Fuzzy Ensemble: score-based aggregation with fuzzification

10

Joinpoint Regression

Joinpoint++: piecewise linear trend with WBIC model selection

Standard Workflow – Change-Point Pipeline

```
library(daltoolbox)
library(harbinger)
```

```
# 1. Load the dataset
data(examples_changepoints)
dataset <- examples_changepoints$simple
# or: $complex
```

```
# 2. Visualize the raw series
har_plot(harbinger(), dataset$serie)
```

```
# 3. Instantiate and fit the model
# <- detailed in each following slide
model <- hcp_<method>(...)
model <- fit(model, dataset$serie)
```

```
# 4. Run detection
detection <- detect(model, dataset$serie)
```

```
# 5. Inspect detected change points
print(detection |>
  dplyr::filter(event == TRUE))
```

```
# 6. Evaluate against ground truth
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
```

```
# 7. Plot series with detections
har_plot(model, dataset$serie,
  detection, dataset$event)
```

```
# 8. Plot residuals / score (optional)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Step by step

- `examples_changepoints`: built-in benchmark datasets (`$simple`, `$complex`, etc.)
- `har_plot(harbinger(), ...)`: visualizes the raw series before fitting
- `hcp_<method>(...)`: instantiates the detector — specific parameters in each slide
- `fit()`: trains the model on the full series
- `detect()`: returns a data frame with `idx`, `event`, and `type` columns
- `evaluate()`: compares detections against ground-truth labels (confusion matrix)
- `har_plot(model, ...)`: overlays detected change points on the series

📌 **Key Concept:** This pipeline is reused across all examples. Only the `hcp_<method>(...)` block changes between slides — the rest remains identical.

Single Breakpoint – AMOC

AMOC (**At Most One Change**) is the foundational change-point detection method. It finds the single most statistically significant structural break by minimizing a cost function over all possible split points. It assumes at most one change exists in the series.

Best for: series where a single dominant regime shift is expected.

Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <- examples_changepoints$simple

# Instantiate AMOC
model <- hcp_amoc()

# --- follow the Standard Workflow ---

## Detection result:
## idx event      type
## 389 TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100
```

Step-by-step

- `hcp_amoc()` requires no parameters — it searches all split points automatically
- `fit()` computes the cost function over the full series
- Detected index **389** — a perfect result: **1 TP, 0 FP**
- Use when you expect **at most one** structural break

 AMOC is the simplest baseline. If your series has more than one break, use BinSeg or PELT instead.

Multiple Breakpoints – Binary Segmentation (BinSeg)

BinSeg is a **greedy recursive algorithm** that extends AMOC to detect multiple structural changes, recursively splitting segments until a maximum number of breakpoints **Q** or a cost threshold is reached. At each step it applies AMOC to the current segment and keeps the best split.

Mathematical Intuition

At each step, find the optimal split point by minimizing the total cost across both segments:

$$\tau^* = \arg \min_{\tau} [\mathcal{C}(x_{1:\tau}) + \mathcal{C}(x_{\tau+1:n})]$$

Split at τ^* , then repeat independently on each sub-segment until the stopping criterion is met.

Best for: series with several well-separated breaks where an approximate (not globally optimal) solution is acceptable. Uses the `complex` dataset from `examples_changepoints`.

```
# Use the complex dataset
dataset <-
examples_changepoints$complex

# Instantiate BinSeg
# Q = maximum number of change points
model <- hcp_binseg(Q = 10)

# --- follow the Standard Workflow ---

## Confusion matrix (example):
## detection TRUE FALSE
##   TRUE  4   6
##   FALSE 0 490
```

Step-by-step

- `hcp_binseg(Q = 10)` sets the maximum number of breakpoints to search for
- Recursively applies AMOC to each sub-segment
- **Greedy:** not guaranteed to find the globally optimal segmentation
- Use **PELT** if you need the exact optimal solution

📌 **Key Concept:** BinSeg trades optimality for speed. It is a good first choice when Q is known approximately.

Multiple Breakpoints – Optimal Partitioning (PELT)

PELT is an exact change-point detection algorithm that finds the globally optimal segmentation of a time series under a penalized cost framework. Unlike BinSeg, it guarantees the globally optimal solution by using dynamic programming with a pruning step that discards candidate split points that can never be optimal, making it linear in the number of change points.

Mathematical Intuition

PELT minimizes the penalized total cost over all possible segmentations:

$$\text{minimize over all } \tau : \sum \text{ of } \mathcal{C}(x_{\tau_{i-1}+1:\tau_i}) + \beta \cdot m$$

where $\mathcal{C}$ is the within-segment cost, β is the penalty per breakpoint, and m is the number of breakpoints. The pruning rule eliminates candidate points τ whenever:

$$\mathcal{C}(x_{\tau:t}) + \beta \geq \mathcal{C}(x_{\tau:s}) + \mathcal{C}(x_{s:t}) \quad \text{for all future } t$$

Best for: series where the number of breaks is unknown and an exact solution is required. Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <- examples_changepoints$simple

# Instantiate PELT
# (no Q needed — penalty auto-selects
breaks)
model <- hcp_pelt()

# --- follow the Standard Workflow ---

## Detection result:
## idx event    type
## 50  TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0  100
```

Step-by-step

- `hcp_pelt()` requires no `Q` parameter — it determines the number of breaks automatically via penalized cost
- Uses dynamic programming with pruning for linear-time complexity
- **Globally optimal:** guaranteed to find the best segmentation
- Penalty controls the trade-off between fit and number of breaks

 **Key Concept:** PELT is the go-to method when the number of breaks is unknown and correctness matters more than speed.

Structural Break Test – Chow

The Chow test (1960) detects a single structural break in a linear regression model by splitting the time series at a candidate breakpoint and comparing the fit of two separate regressions against a single pooled regression. It uses an F-statistic to test whether the regression coefficients differ significantly across the two segments.

Mathematical Intuition

The Chow test computes an F-statistic comparing pooled vs. segmented residuals:

$$F = \frac{(RSS_p - (RSS_1 + RSS_2))/k}{(RSS_1 + RSS_2)/(n - 2k)}$$

where RSS_p is the pooled residual sum of squares, RSS_1 and RSS_2 are the segment residuals, k is the number of parameters, and n is the total number of observations.

Best for: series modeled by linear regression where a single structural break in the relationship is suspected. Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <- examples_changepoints$simple

# Instantiate Chow
model <- hcp_chow()

# --- follow the Standard Workflow ---

## Detection result:
## idx event    type
## 50  TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100
```

Step-by-step

- `hcp_chow()` requires no parameters — it tests all candidate breakpoints
- Fits two separate regressions and compares them via an F-statistic
- Detects a **single** structural break (like AMOC, but regression-based)
- Sensitive to changes in **slope or intercept** of the linear trend

❏ **Key Concept:** Chow is the regression-based counterpart to AMOC. Use it when the series follows a linear model and you want a hypothesis-test interpretation.

Structural Break Test – GFT

GFT (Zeileis et al., 2002) extends the Chow test by monitoring the stability of regression parameters over the entire time span, rather than testing a single candidate breakpoint. It tracks a fluctuation process derived from recursive or moving-window residuals and flags a change point when the process exceeds a critical boundary.

Best for: series where the break location is unknown and a global monitoring approach is preferred over a single-point test. Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <- examples_changepoints$simple

# Instantiate GFT
model <- hcp_gft()

# --- follow the Standard Workflow ---

## Detection result:
## idx event    type
## 50 TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100
```

Step-by-step

- `hcp_gft()` requires no parameters — it monitors the full fluctuation process
- Tracks cumulative residuals from a recursive regression fit
- Flags a change when the fluctuation process crosses a critical boundary
- More robust than Chow when the break location is unknown

 **Key Concept:** GFT is the global monitoring counterpart to Chow. It is preferred when you do not have a prior candidate breakpoint location.

ChangeFinder – CF-LR

CF-LR (Yamanishi & Takeuchi, 2002) fits a linear autoregressive model over a sliding window and scores each observation by its prediction error. Scores are smoothed a second time to reduce noise, and a threshold flags persistent deviations as change points. It detects structural shifts through predictive surprise: the model continuously forecasts the next value, and residual deviations are smoothed and thresholded over time to expose underlying regime changes.

Best for: series with gradual or abrupt mean shifts detectable by linear prediction error. Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <- examples_changepoints$simple

# Instantiate CF-LR
# sw_size controls the window size
model <- hcp_cf_lr(sw_size = 10)

# --- follow the Standard Workflow ---

## Detection result:
## idx event    type
## 50 TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE  1    0
##   FALSE 0  100
```

Step-by-step

- `hcp_cf_lr(sw_size = 10)` sets the sliding window size for the linear model
- Scores each point by its linear prediction error
- Applies a second smoothing pass to reduce noise before thresholding
- `sw_size` controls sensitivity: smaller windows react faster but produce more false positives

📌 **Key Concept:** CF-LR is the simplest ChangeFinder variant. Use CF-ARIMA or CF-ETS for series with stronger temporal structure.

ChangeFinder – CF-ARIMA

CF-ARIMA replaces the linear regression in ChangeFinder with an ARIMA model, capturing temporal autocorrelation and non-stationarity. By modeling the autoregressive and moving-average structure of the data, it produces more accurate predictions for autocorrelated series, making the residual score more sensitive to genuine regime changes rather than normal autocorrelation patterns.

Best for: series with significant autocorrelation or non-stationarity where CF-LR produces too many false positives. Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <-
examples_changepoints$simple

# Instantiate CF-ARIMA
model <- hcp_cf_arima(sw_size = 10)

# --- follow the Standard Workflow ---

## Detection result:
## idx event    type
## 50 TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100
```

Step-by-step

- `hcp_cf_arima(sw_size = 10)` sets the sliding window for ARIMA fitting
- ARIMA captures autocorrelation and trend, producing cleaner residuals
- More computationally expensive than CF-LR due to ARIMA fitting per window
- Preferred over CF-LR when the series has strong temporal dependencies

 **Key Concept:** CF-ARIMA is the most statistically rigorous ChangeFinder variant for autocorrelated series. Use CF-ETS when the series also has a seasonal component.

ChangeFinder – CF-ETS

CF-ETS substitutes the forecasting model in ChangeFinder with an ETS (Error, Trend, Seasonality) model. ETS uses exponential smoothing to decompose the series into level, trend, and seasonal components, assigning exponentially decaying weights to past observations. This makes it particularly effective for series with trend or seasonal patterns, where ARIMA may overfit or CF-LR may miss the structure.

Best for: series with trend or seasonal components where exponential smoothing provides a better baseline forecast. Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <- examples_changepoints$simple

# Instantiate CF-ETS
model <- hcp_cf_ets(sw_size = 10)

# --- follow the Standard Workflow ---

## Detection result:
## idx event    type
## 50 TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100
```

Step-by-step

- `hcp_cf_ets(sw_size = 10)` sets the sliding window for ETS fitting
- ETS decomposes the series into level, trend, and seasonal components
- Exponential smoothing gives more weight to recent observations
- Best suited for series with trend or seasonal structure

📌 **Key Concept:** CF-ETS completes the ChangeFinder family. Choose the variant whose forecasting model best matches your series structure: LR → ARIMA → ETS.

Volatility Change – GARCH

GARCH (Generalized Autoregressive Conditional Heteroskedasticity) models the conditional variance of a time series rather than its mean. Unlike Chow or CF-LR, which detect shifts in level or trend, GARCH detects changes in volatility regimes — periods where the variance of the series increases or decreases persistently. It fits a GARCH model over a sliding window and scores each observation by the deviation of its squared residual from the expected conditional variance.

Best for: financial or economic series where volatility clustering is the primary signal of a regime change. Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <- examples_changepoints$simple

# Instantiate GARCH-based detector
model <- hcp_garch()

# --- follow the Standard Workflow ---

## Detection result:
## idx event    type
## 50  TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE   1    0
##  FALSE   0  100
```

Step-by-step

- `hcp_garch()` requires no parameters — it fits a GARCH(1,1) model by default
- Scores each point by the deviation of its squared residual from expected variance
- Detects **volatility regime shifts**, not mean shifts
- Particularly useful for financial time series with heteroskedasticity

 **Key Concept:** GARCH is the only method in this tutorial that targets variance changes rather than mean changes. Use it when the signal is in the volatility, not the level.

Local Window – SCP

SCP (Ogasawara et al., 2010) detects change points by comparing linear regression fits within a sliding window. For each window position, it fits two models: one using all observations, and one excluding the central observation of the window. A large difference in fit quality around the center signals the likely location of a change. SCP relies on local contrast — it compares the fit of the full window against the fit without the central point, making it highly sensitive to sharp, well-localized structural breaks.

Best for: series with sharp, well-localized breaks where local contrast is a reliable signal. Uses the `simple` dataset from `examples_changepoints`.

```
# Use the simple dataset
dataset <- examples_changepoints$simple

# Instantiate SCP
# sw = sliding window size
model <- hcp_scp(sw = 30)

# --- follow the Standard Workflow ---

## Detection result:
## idx event    type
## 50 TRUE changepoint

## Confusion matrix:
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100
```

Step-by-step

- `hcp_scp(sw = 30)` defines a sliding window of 30 observations
- Compares linear regression fits *with* and *without* the central observation
- Large deviations around the center signal the likely location of a change
- Detected exactly index **50** — a perfect result: **1 TP, 0 FP**
- The window size `sw` controls sensitivity: smaller windows pick up more localized changes

❏ **Key Concept:** SCP is ideal for sharp, well-localized breaks. The window size is the key tuning parameter — start with `sw = 30` and adjust based on the expected break width.

Adaptive Waypoint – Feed-Forward (ED)

Waypoint combines an autoencoder with a bilateral CUSUM supervisor to detect regime changes. The autoencoder learns the current regime from sliding windows; the window reconstruction error is standardized on a recent buffer, and a bilateral CUSUM validates persistent deviations. After confirmation, the model is retrained on the new regime. This first variant uses a plain feed-forward autoencoder as the baseline encoder architecture.

Best for: complex multi-regime series where reconstruction error from a neural autoencoder provides a more expressive change signal than linear methods. Uses the complex dataset from `examples_changepoints`. Requires `daltoolboxdp`.

```
library(daltoolboxdp)
dataset <- examples_changepoints$complex

# Instantiate Waypoint with feed-forward encoder
model <- hcp_waypoint(
  input_size  = 12,
  encode_size = 4,
  warmup      = 60,
  retrain_size = 30,
  buffer_size = 40,
  k_factor    = 0.35,
  h_low       = 3.5,
  h_high      = 6,
  prob_tau    = 0.997,
  epochs_init = 100,
  epochs_retrain = 100,
  encoderclass = autoenc_ed
)
set_example_seed()

# --- follow the Standard Workflow ---

## Detection result (sample):
## idx event   type
## 116 TRUE changepoint
## 188 TRUE changepoint
## 303 TRUE changepoint
```

Step-by-step

- `encoderclass = autoenc_ed` selects the feed-forward autoencoder (baseline)
- `input_size = 12`: sliding window length fed to the autoencoder
- `encode_size = 4`: bottleneck dimension
- `warmup = 60`: observations before the CUSUM supervisor activates
- `h_low / h_high`: bilateral CUSUM thresholds for change confirmation
- After confirmation, the model retrain on the new regime (`retrain_size`)

📌 **Key Concept:** Waypoint is the baseline of the architecture comparison. The same parameters are reused in the LSTM and Conv variants — only `encoderclass` changes.

Adaptive Waypoint – LSTM

This variant replaces the feed-forward encoder in Waypoint with an LSTM autoencoder, emphasizing temporal order inside the sliding window. LSTM cells maintain a hidden state across the window, allowing the encoder to capture sequential dependencies that a feed-forward network would miss. This makes the reconstruction error more sensitive to changes in the temporal dynamics of the series, rather than just its statistical distribution.

Best for: series where the temporal ordering within the window carries important information about the regime. Uses the `complex` dataset. Requires `daltoolboxdp`.

```
library(daltoolboxdp)
dataset <- examples_changepoints$complex

# Instantiate Waypoint with LSTM encoder
# Only encoderclass changes from ED baseline
model <- hcp_waypoint(
  input_size  = 12,
  encode_size = 4,
  warmup      = 60,
  retrain_size = 30,
  buffer_size = 40,
  k_factor    = 0.35,
  h_low       = 3.5,
  h_high      = 6,
  prob_tau    = 0.997,
  epochs_init = 100,
  epochs_retrain = 100,
  encoderclass = autoenc_lstm # <-- LSTM
)
set_example_seed()

# --- follow the Standard Workflow ---
```

Step-by-step

- `encoderclass = autoenc_lstm` is the only change from the ED baseline
- LSTM cells process the window sequentially, preserving temporal order
- More expressive than ED for series with sequential dependencies
- Higher computational cost than ED due to recurrent connections
- Compare results with ED and Conv to assess the impact of temporal bias

📌 **Key Concept:** LSTM emphasizes temporal order within the window. If the regime change manifests as a shift in sequential dynamics, LSTM will outperform ED.

Adaptive Waypoint – Convolutional (Conv)

This variant uses a convolutional autoencoder, emphasizing local patterns inside the sliding window. Convolutional filters scan the window for recurring local motifs, making the encoder sensitive to changes in the local shape or texture of the series rather than its global sequential structure. This completes the Waypoint architecture comparison: ED (generic baseline), LSTM (temporal order), Conv (local patterns). Best for: series where regime changes manifest as shifts in local waveform patterns or textures. Uses the `complex` dataset. Requires `daltoolboxdp`.

```
library(daltoolboxdp)
dataset <- examples_changepoints$complex

# Instantiate Waypoint with Conv encoder
# Only encoderclass changes from ED baseline
model <- hcp_waypoint(
  input_size  = 12,
  encode_size = 4,
  warmup     = 60,
  retrain_size = 30,
  buffer_size = 40,
  k_factor   = 0.35,
  h_low      = 3.5,
  h_high     = 6,
  prob_tau   = 0.997,
  epochs_init = 100,
  epochs_retrain = 100,
  encoderclass = autoenc_conv # <-- Conv
)
set_example_seed()

# --- follow the Standard Workflow ---
```

Step-by-step

- `encoderclass = autoenc_conv` is the only change from the ED baseline
- Convolutional filters detect local patterns within the window
- Sensitive to changes in local shape or texture, not global sequential order
- Completes the Waypoint architecture comparison: **ED → LSTM → Conv**
- Differences in detections across the three variants reflect encoder bias

📌 **Key Concept:** Conv emphasizes local patterns. Compare all three Waypoint variants on your data to determine which encoder bias best matches your regime change structure.

Bayesian Online Monitoring – BOCPD

BOCPD (Adams & MacKay, 2007) tracks change evidence sequentially by updating a posterior distribution over run length — the number of observations since the last change point. At each step, it computes the probability that a new regime has started, given all observations so far. In `hcp_bocpd()`, that evidence is summarized as a normalized score and thresholded to produce final change-point candidates. Best for: online monitoring scenarios where change evidence should accumulate probabilistically over time, rather than being tested at a fixed candidate point. Uses the `complex` dataset from `examples_changepoints`. Requires the `ocp` package.

```
# Requires the ocp package
# install.packages("ocp")
dataset <- examples_changepoints$complex

# Instantiate BOCPD
model <- hcp_bocpd(
  hazard = 100,
  dist = "gaussian",
  threshold = 0.5
)

# --- follow the Standard Workflow ---
# (wrap in requireNamespace check)

if (requireNamespace("ocp", quietly = TRUE)) {
  set_example_seed()
  # fit / detect / evaluate / plot
  # as in the Standard Workflow
}

## Detection result (sample):
## idx event    type
## 102 TRUE changepoint
## 312 TRUE changepoint
```

Step-by-step

- `hazard = 100`: prior expected run length between change points
- `dist = "gaussian"`: observation model used by the `ocp` package
- `threshold = 0.5`: maps normalized changepoint evidence into final decisions
- The method is wrapped in `requireNamespace("ocp")` since `ocp` is an optional dependency
- Higher `hazard` → expects fewer, longer regimes; lower → more frequent changes

 **Key Concept:** BOCPD is the only fully probabilistic online method in this tutorial. It is ideal when you need calibrated uncertainty about change-point locations.

Ensemble Support – Fuzzy Ensemble

The fuzzy ensemble `har_ensemble_fuzzy()` aggregates the outputs of multiple change-point detectors as scores, applies temporal fuzzification around nearby detections, and can use non-maximum suppression (NMS) to keep the most representative event in a local neighborhood. This allows complementary detectors to reinforce each other near true change points while suppressing isolated false positives. The ensemble members are chosen to provide complementary signals: `hcp_scp(sw=30)` and `hcp_chow()` give exact hits; `hcp_cf_arima(sw_size=10)` reinforces the neighborhood under fuzzy aggregation. Best for: combining multiple detectors to improve precision and reduce false positives. Uses the `complex` dataset.

```
dataset <- examples_changepoints$complex

# Instantiate the fuzzy ensemble
model <- har_ensemble_fuzzy(
  hcp_scp(sw = 30),
  hcp_chow(),
  hcp_cf_arima(sw_size = 10)
)
set_example_seed()

# Detection uses extra parameters:
detection <- detect(model, dataset$serie,
  time_tolerance = 8,
  use_nms = TRUE)

## Detection result:
## idx event type
## 200 TRUE

# Plot ensemble score
har_ensemble_plot(detection, dataset$serie)
```

Step-by-step

- Pass any number of `hcp_*` detectors to `har_ensemble_fuzzy()`
- `time_tolerance = 8`: detections within 8 steps of each other are fuzzified together
- `use_nms = TRUE`: keeps only the highest-scoring detection in each neighborhood
- Ensemble members should be **complementary** — different methods, different biases
- The fused score is plotted with `har_ensemble_plot()`

📌 **Key Concept:** Ensembles are the most robust approach when no single method dominates. Combine detectors with different assumptions to cover a wider range of change types.

Univariate Drift – Page-Hinkley

Page-Hinkley (Page, 1954) is a sequential test for persistent mean shifts. It monitors the cumulative deviation of each observation from the running mean and flags a change point when the cumulative score exceeds a threshold. In Harbinger it is used as a univariate change-point detector for virtual drift in a single numeric series. Unlike batch methods (AMOC, PELT), Page-Hinkley operates online — it processes one observation at a time and can flag changes as they occur. Best for: online monitoring of univariate series where a persistent upward or downward mean shift is the primary signal. Uses the `complex` dataset from `examples_changepoints`.

```
dataset <- examples_changepoints$complex
```

```
# Instantiate Page-Hinkley
model <- hcp_page_hinkley(
  min_instances = 30,
  delta        = 0.005,
  threshold     = 3,
  alpha        = 0.999
)
set_example_seed()
```

```
# --- follow the Standard Workflow ---
```

```
## Detection result (sample):
## idx event   type
## 28 TRUE changepoint
## 57 TRUE changepoint
## 115 TRUE changepoint
```

Step-by-step

- `min_instances = 30`: minimum observations before the test activates
- `delta = 0.005`: allowable mean deviation (sensitivity parameter)
- `threshold = 3`: cumulative score threshold for flagging a change
- `alpha = 0.999`: forgetting factor — controls how fast old deviations decay
- Operates **online**: processes one observation at a time
- Note: may produce multiple detections per true change (sequential nature)

📌 **Key Concept:** Page-Hinkley is the simplest online drift detector. It is fast and interpretable, but sensitive to the threshold and delta parameters.

Univariate Drift – KSWIN

KSWIN (Kolmogorov-Smirnov Windowing, Raab et al., 2020) is a window-based sequential detector for distributional change. It compares an early sample with the most recent observations inside a sliding window using the Kolmogorov-Smirnov test, and flags a change point when the two distributions differ significantly. Unlike Page-Hinkley, which tests for mean shifts, KSWIN is sensitive to any distributional change — including shifts in variance, shape, or higher moments. In Harbinger it is restricted to one numeric series so that the output can be interpreted directly as virtual drift on the observed signal. Best for: series where the distributional change is not limited to the mean. Uses the `complex` dataset from `examples_changepoints`.

```
dataset <-
examples_changepoints$complex

# Instantiate KSWIN
model <- hcp_kswin(
  window_size = 100,
  stat_size   = 30,
  alpha      = 0.005
)
set_example_seed()

# --- follow the Standard Workflow ---

## Detection result:
## idx event   type
## 213 TRUE changepoint
## 318 TRUE changepoint
## 453 TRUE changepoint
```

Step-by-step

- `window_size = 100`: total sliding window length
- `stat_size = 30`: size of the recent sub-sample compared against the early sub-sample
- `alpha = 0.005`: KS test significance level for flagging a change
- Detects **any distributional shift**, not just mean changes
- More general than Page-Hinkley but requires a larger window to be reliable

📌 **Key Concept:** KSWIN is the distribution-free counterpart to Page-Hinkley. Use it when you suspect variance or shape changes, not just mean shifts.

Joinpoint Regression++ – hcp_joinpoint

`hcp_joinpoint()` searches for a piecewise linear representation of the series. It evaluates models with 0 to `k_max` joinpoints, computes BIC, BIC3, and WBIC, and keeps the model with the smallest WBIC score. This implementation uses dynamic programming for global segment optimization, so it does not rely on brute-force breakpoint enumeration. It is described as Joinpoint Regression++ to distinguish it from the original permutation-test workflow in the reference literature (Kim et al., 2000). Best for: series where the main evidence of change is a shift in slope or trend level rather than an isolated spike. Uses the `complex` dataset from `examples_changepoints`.

```
dataset <- examples_changepoints$complex

# Instantiate Joinpoint Regression++
model <- hcp_joinpoint(
  min_between = 20,
  min_end     = 20,
  k_max       = 1,
  log_transform = FALSE
)
set_example_seed()

# --- follow the Standard Workflow ---

## Model comparison (WBIC selects k=1):
## k    RSS    WBIC
## 1 0 9010.699 2.916418
## 2 1  911.836 0.661554 <-- selected

## Detection result:
## idx event    type
## 201  TRUE changepoint
```

Step-by-step

- `k_max = 1`: maximum number of joinpoints to consider
- `min_between = 20`: minimum observations between two joinpoints
- `min_end = 20`: minimum observations at the start and end of the series
- Model selection uses **WBIC** (Widely Applicable BIC) to choose the best `k`
- `log_transform = FALSE`: set to `TRUE` for count or rate data (e.g., cancer rates)
- Detected index **201** — close to the true change point

📌 **Key Concept:** Joinpoint is ideal for trend-change detection in epidemiological or economic data. It finds where the slope of the series changes, not just the level.



Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

